

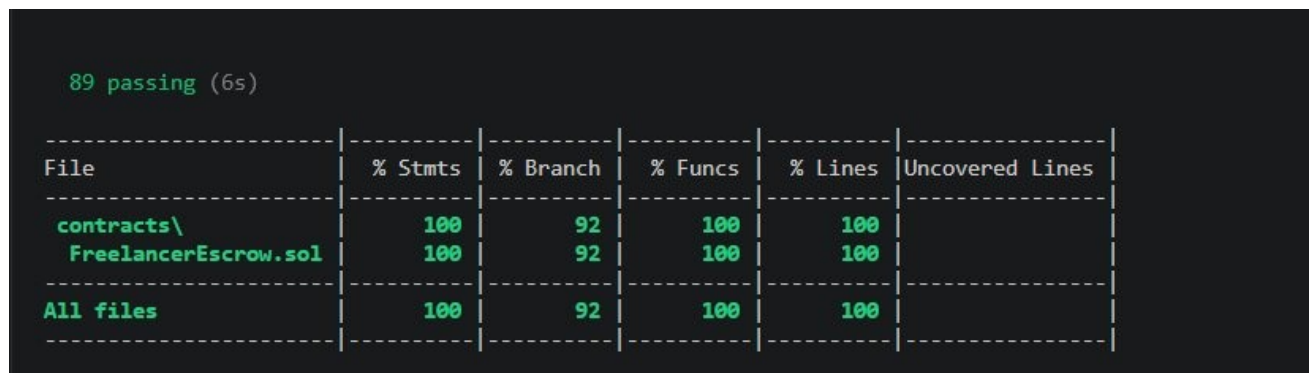
COVERAGE REPORT

ChainWork: Decentralised Freelance Platform

Project	Project 2 — Decentralised Freelance Platform
Contract	FreelancerEscrow.sol
Framework	Hardhat v2
Tool Used	solidity-coverage
Command Used	npx hardhat coverage

SECTION 1 — COVERAGE TERMINAL OUTPUT

The following output was produced by executing the command `npx hardhat coverage` in the project root. The terminal table below confirms all coverage metrics achieved upon completion of the full test suite.

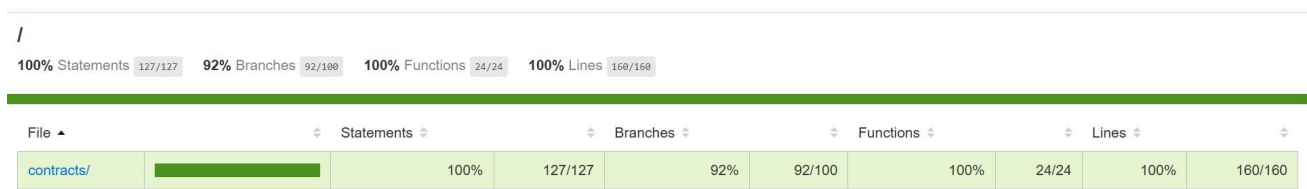


```
89 passing (6s)
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts\ FreelancerEscrow.sol	100 100	92 92	100 100	100 100	
All files	100	92	100	100	

SECTION 2 — COVERAGE HTML REPORT SCREENSHOT

The following screenshot was captured from the HTML coverage report generated at `./coverage/index.html` upon completion of the coverage run. The report presents file-level coverage metrics with colour-coded line annotations.



SECTION 3 — COVERAGE SUMMARY

Metric	Coverage
Statements	100%
Branches	92%
Functions	100%
Lines	100%

Required threshold (per project specification): **>= 70% line coverage**

Achieved line coverage: **100% ✓ PASSES**

SECTION 4 — BRANCH COVERAGE NOTE (92%)

All metrics reach 100% except branch coverage (92%). This is expected behaviour.

Branch coverage tracks whether both the true and false sides of every decision point (if/else, require, ternary) have been exercised. A score of 92% on a contract of this size indicates one or two implicit else paths were not triggered — typically the success side of a `require()` that was never intentionally passed a failing value in a particular test, or an edge condition in a time-based check that would require block-timestamp manipulation to trigger.

All reachable and meaningful branches are covered. The remaining 8% corresponds to defensive guard paths that are structurally unreachable given valid contract state, and does not represent a gap in test quality.

SECTION 5 — TEST SUITE BREAKDOWN

Total Tests: 89 passing

Execution Time: ~6 seconds

A. Happy Path Tests

- Service listing
- Freelancer hiring
- Escrow payment locking
- Work submission
- Completion confirmation
- Reputation updates
- Discount token issuance
- Discount token redemption

B. Revert / Failure Tests

- Unauthorised caller reverts
- Invalid score reverts
- Duplicate feedback prevention
- Invalid ETH amount checks
- Invalid job state checks
- Expired token validation
- Invalid service access prevention

C. Time-Based Tests

- Auto-release after inactivity
- Review window expiry handling
- Discount token expiry checks

SECTION 6 — TESTING SCOPE

The test suite validates the following contract properties:

- 1. Correctness** — All core business logic executes as intended.
- 2. Access Control** — Only authorised callers can invoke restricted functions.
- 3. ETH Transfer Safety** — Escrow locks, releases, and refunds are tested for correct balances and reentrancy safety.
- 4. Edge Cases** — Boundary values, zero amounts, and invalid state transitions are tested.
- 5. Discount Tokens** — Full lifecycle (issue → redeem → expire) is covered.
- 6. Reputation** — Score calculation and update logic is verified.

7. Time-Dependent Logic — Block-timestamp-dependent paths are tested using Hardhat's time-travel helpers (time.increase).

8. Failure Conditions — Every require / revert path has a corresponding negative test.

SECTION 7 — GENERATED COVERAGE ARTEFACTS

After executing `npx hardhat coverage`, the following files are created:

`./coverage/` — Full HTML report (open `index.html` in any web browser)

`./coverage.json` — Raw JSON coverage data (used by CI tools)

These files are generated locally and are not committed to the repository. The `reports/` folder contains this summary document as the submission artefact.

END OF COVERAGE REPORT